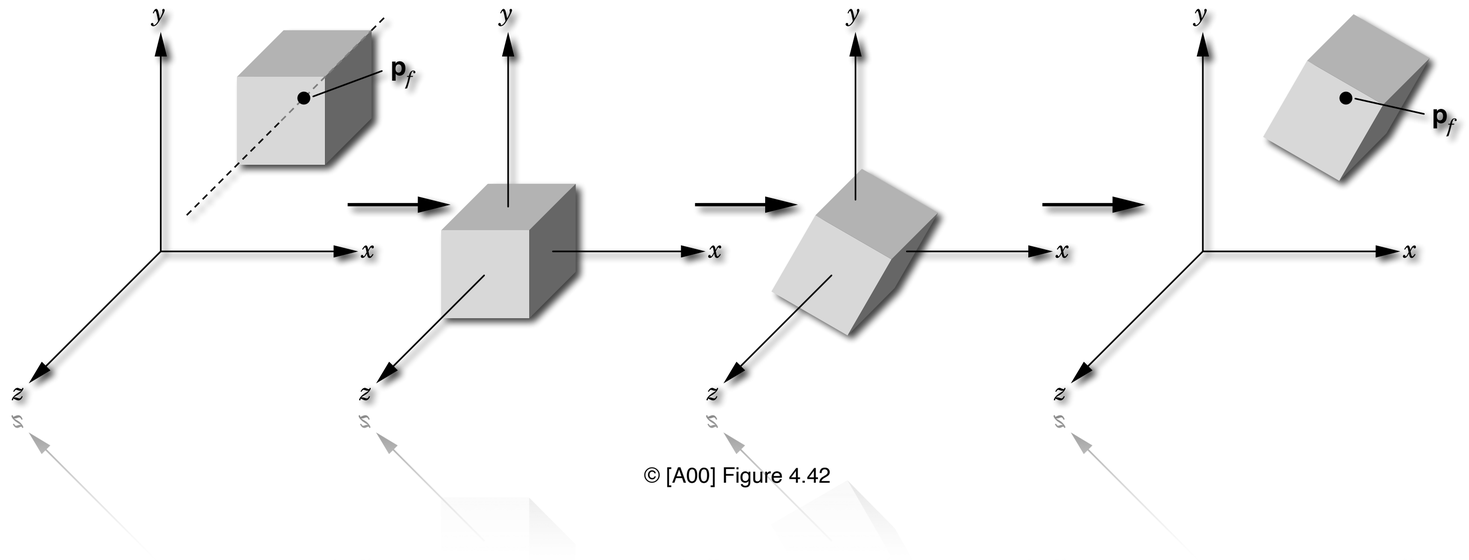
Advanced Transformations and Implementation



© [A00] Figure 4.42

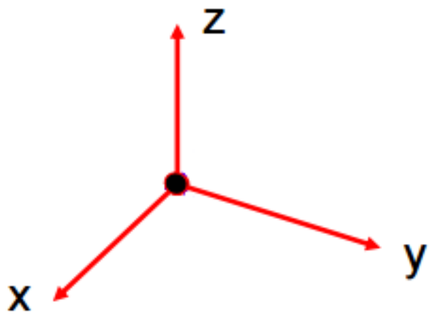
Special Orthogonal Rotation

- The row and column vectors of the upper-left 3x3 submatrix are mutually perpendicular unit vectors
  - determinant of 3x3 submatrix equal to 1
  - upper-left 3x3 submatrix formed by any arbitrary sequence of rotations is special orthogonal

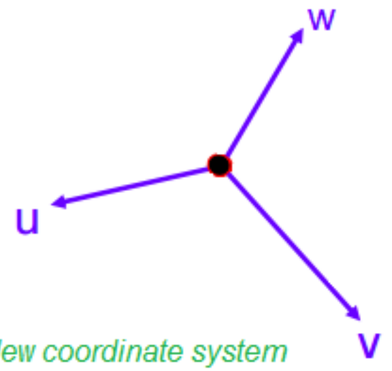
$$\begin{bmatrix} r_{11} & r_{12} & r_{13} & 0 \\ r_{21} & r_{22} & r_{23} & 0 \\ r_{31} & r_{32} & r_{33} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- A rotation is defined by the unit-vectors of the coordinate axes directions of the new (rotated) coordinate system, expressed in the old coordinate system
  - the new coordinate system's unit axis vectors define the rows of the rotation matrix R
  - e.g. new x-axis direction will be $\boldsymbol{x}_{\text{new}} = [r_{11} \;\; r_{12} \;\; r_{13}]$

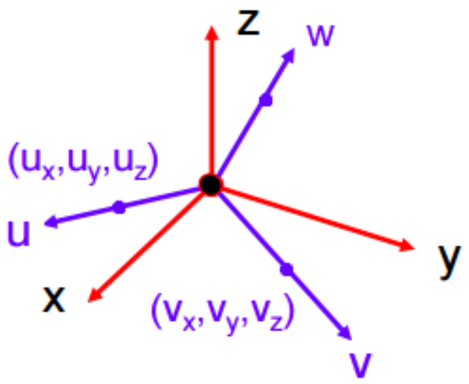
Special Orthogonal Rotation



Old coordinate system



New coordinate system



$$R = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ w_x & w_y & w_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Source: https://stackoverflow.com/questions/31285396/change-from-one-cartesian-3d-co-ordinate-system-to-another-by-translation-and-ro

Transformation Task

(a) Initial position

(b) Final position

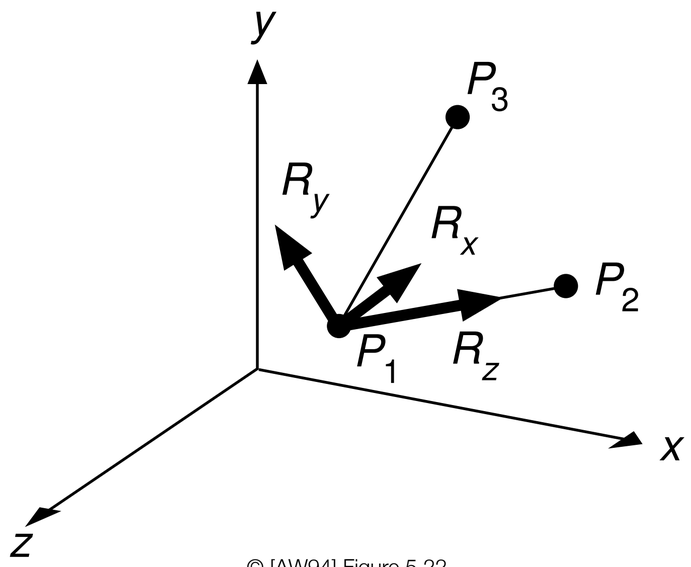
- Transform P_1 to origin, P_2 onto z-axis and P_3 into yz-plane using rigid body transforms
- **Second approach**: exploit property of *special orthogonal* transformation matrices
  1. translate origin to P_1 (i.e. $T(\text{-}x_1, \text{-}y_1, \text{-}z_1)$)
  2. set unit row vectors of composite rotation matrix R such as to transform the principal axes as required

Transformation Task 1b

$$R = \begin{bmatrix} r_{1_x} & r_{2_x} & r_{3_x} \\ r_{1_y} & r_{2_y} & r_{3_y} \\ r_{1_z} & r_{2_z} & r_{3_z} \end{bmatrix}$$

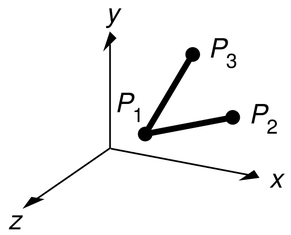
- Rotation by R transforms coordinate system such that the new principal axis directions align with row the vectors
  $R_{\mathrm{x}} = [r_{1x}, r_{2x}, r_{3x}]$, $R_{\mathrm{y}} = [r_{1y}, r_{2y}, r_{3y}]$ and $R_{\mathrm{z}} = [r_{1z}, r_{2z}, r_{3z}]$

- Setup row vectors R_{x}, R_{y} and R_{z} accordingly to the given object transformation task

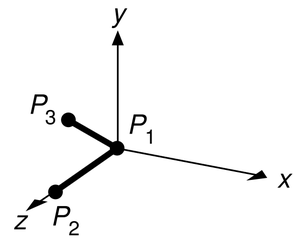


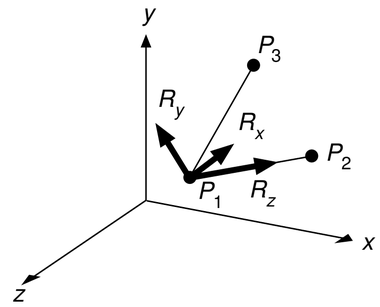
© [AW94] Figure 5.22

Transformation Task 1b



(a) Initial position





(b) Final position

© [AW94] Figures 5.18 and 5.22

- Since P_1P_2 should align with the z-axis, $\boldsymbol{R_z}$ **is simply the unit vector along** $\boldsymbol{P_1P_2}$

$$R_z = \frac{\overrightarrow{P_1P_2}}{\|\overrightarrow{P_1P_2}\|}$$

- R_x must be perpendicular to the plane $P_1P_2P_3$ since it should align with the y,z-principal plane, **hence** $\boldsymbol{R_x}$ **is defined as the normalized cross product of the plane spanning vectors** $\boldsymbol{P_1P_3}$ **and** $\boldsymbol{P_1P_2}$

$$R_x = \frac{\overrightarrow{P_1P_3} \times \overrightarrow{P_1P_2}}{\|\overrightarrow{P_1P_3} \times \overrightarrow{P_1P_2}\|}$$

- Finally $\boldsymbol{R_y}$ **is simply defined by the cross product of** $\boldsymbol{R_z}$ **and** $\boldsymbol{R_x}$

$$R_y = R_z \times R_x$$

Transformation Task 1b

- **Final composite:** transform $M = R \cdot T$ using $T(\text{-}x_1, \text{-}y_1, \text{-}z_1)$ and setting the row vectors $R_x = [r_{1x}, r_{2x}, r_{3x}]$, $R_y = [r_{1y}, r_{2y}, r_{3y}]$ and $R_z = [r_{1z}, r_{2z}, r_{3z}]$ for R

$$M = \begin{bmatrix} r_{1_x} & r_{2_x} & r_{3_x} & 0 \\ r_{1_y} & r_{2_y} & r_{3_y} & 0 \\ r_{1_z} & r_{2_z} & r_{3_z} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot T(-x_1, -y_1, -z_1)$$

Rigid-body Object Placement

- Transformation that orients an object followed by a translation to its new position

$$\begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

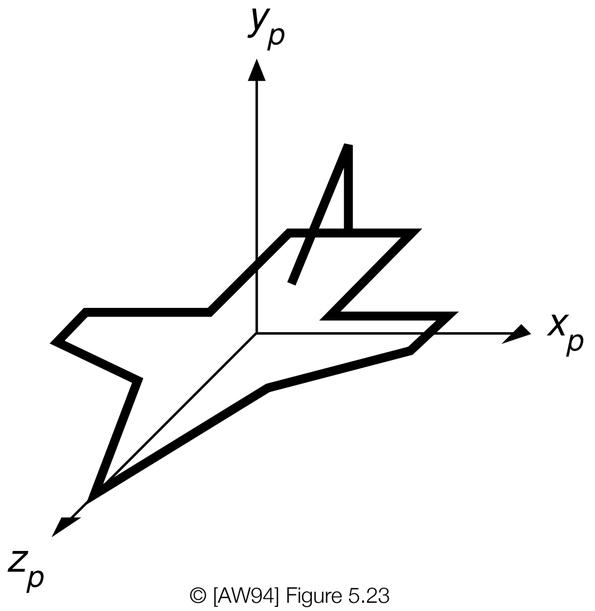
- Its **column vectors** define the new orientations of the originally axis-aligned edges of the object after the rotation, and the last column indicates a final subsequent translation
  - transforms originally axis-aligned edges to align with the matrix's columns $[r_{11} \;\; r_{21} \;\; r_{31}]^{\mathrm{T}}$, $[r_{12} \;\; r_{22} \;\; r_{32}]^{\mathrm{T}}$ or $[r_{13} \;\; r_{23} \;\; r_{33}]^{\mathrm{T}}$
  - followed by translation to position $(t_{\mathrm{x}}, t_{\mathrm{y}}, t_{\mathrm{z}})$

Object Animation

- Animate object along a given path, e.g. simulate navigation route of vehicle
  - object given in 1:1 scale at origin of the modeling coordinate system
  - pointing forward along the z-axis and the up direction being the y-axis

Task:

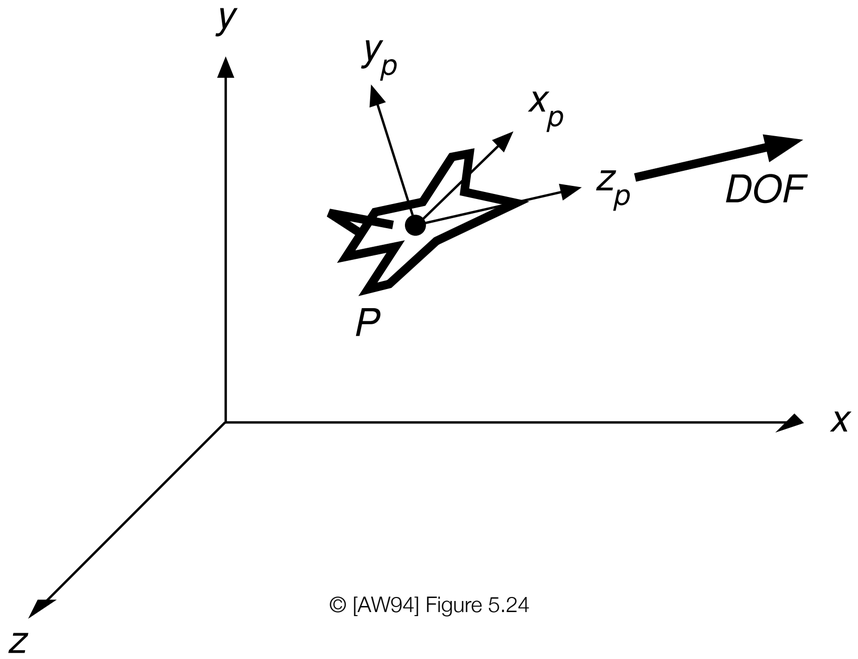
- Place airplane at given location P in world coordinates and orient it to a given direction-of-flight vector DOF
  - P and DOF given by simulation or recorded data
  - the wings of the airplane should stay horizontal in the air



© [AW94] Figure 5.23

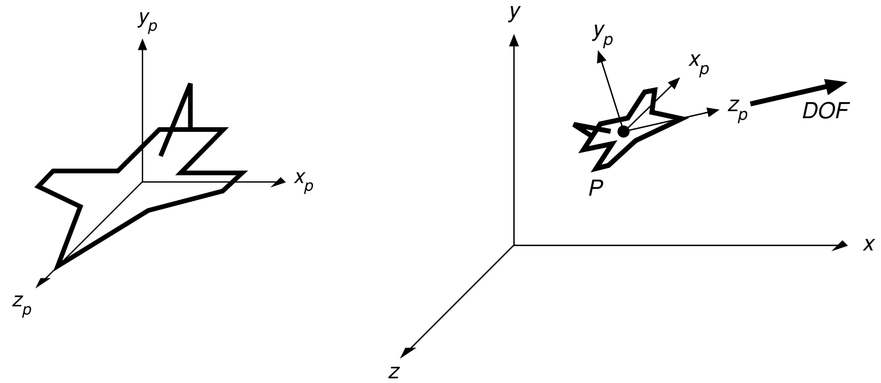
Transformation

- Re-orient airplane by rotation(s) to the given DOF, followed by translation from the origin to P
  - rotation can be found by orienting the model's coordinate system axes x_P, y_P, z_P to the vectors given and derived by the DOF
    - use the rigid body object placement column vector properties
  - translation given by $T(P)$



© [AW94] Figure 5.24

Rotation Matrix



- z_P axis maps to the DOF vector from the animation
- x_P is to be horizontal and perpendicular to DOF, this is achieved by

$$[0,1,0]^T \times DOF$$

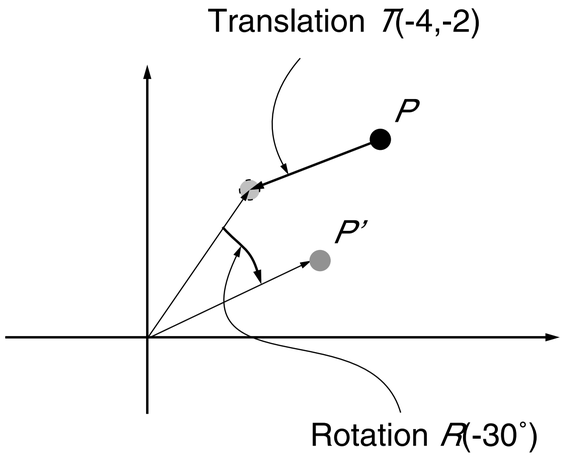
- y_P is perpendicular to both, z_P and x_P which can be achieved by

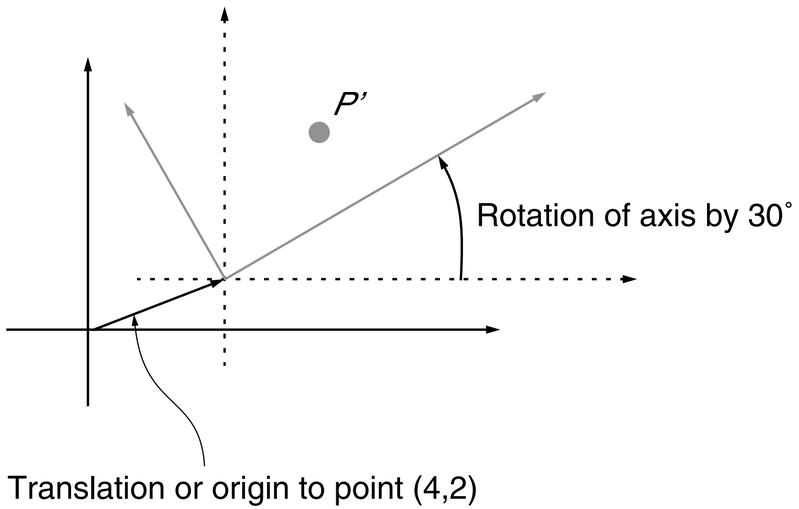
$$z_P \times x_P = DOF \times ([0,1,0]^T \times DOF)$$

- Column vectors of rotation matrix:
  - **column vectors** because
    not the coordinate system
    is changed BUT the object rotated (= inverse)
    - $\|v\|$ denotes normalization of vector to unit length

$$R = \begin{bmatrix} & & & 0 \\ \|y \times \mathrm{DOF}\| & \|\mathrm{DOF} \times (y \times \mathrm{DOF})\| & \|\mathrm{DOF}\| & 0 \\ & & & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Object vs. Coordinate System Transform

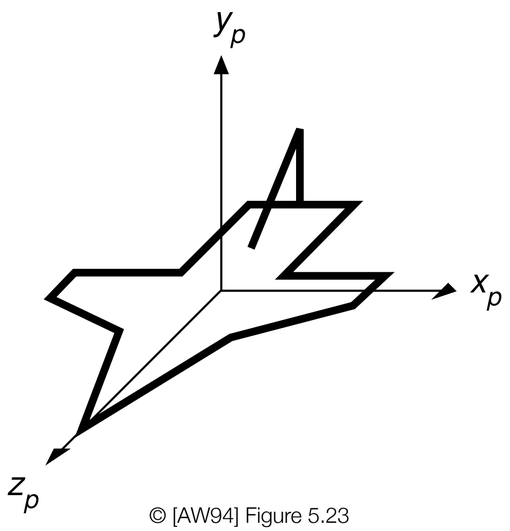




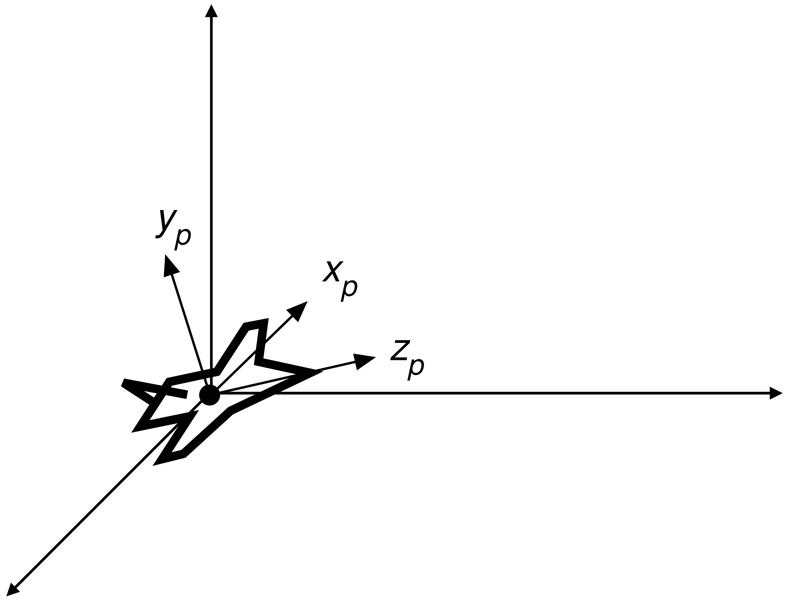
- Transformations applied to objects correspond to an **inverted** transformation of the coordinate system
  - but applied in the **same order**
- Example: object translation and rotation $P' = R(-30^o) \cdot T(-4,-2) \cdot P$
  - $\rightarrow$ moves point closer to origin by $(-4,-2)$ and rotates clockwise by 30^o
  - $\rightarrow$ moves origin closer toward point by $(4,2)$ and rotates axis counterclockwise by 30^o

Example Object / Coordinate System Transformation

- The orientation of the airplane by rotating its forward and upward pointing model coordinate axes to the given DOF vector, and horizontal wing orientation, corresponds to the dual view of object transformations and modifications of the coordinate system axes
  - conceptual modification of coordinate system axes is the inversion of object transformations
  - using columns instead of the rows of the special orthogonal rotation matrix



© [AW94] Figure 5.23

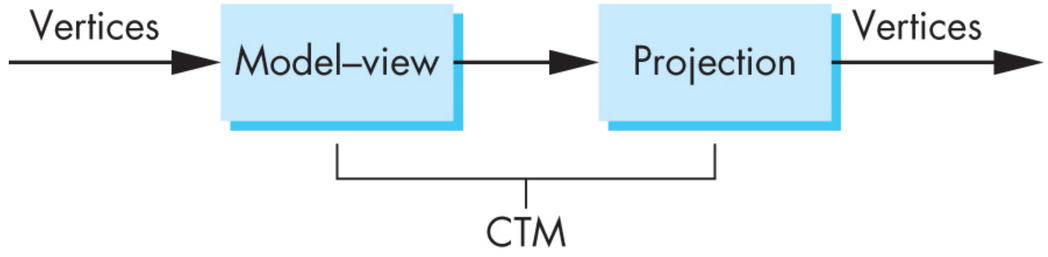


Managing Transformations

- Modern OpenGL applications can choose which frames to use as well as when and where to apply transformations
- The application programmer can freely chose to work in model, world or camera coordinates
  - transformations can be applied by the main program on the CPU or in shaders on the GPU
- Most important pair of transformations to be managed are the model-view and the (perspective) projection transformation
  - model-view transformation brings the representation of geometric objects to the coordinate system of the camera
    - normally an affine transformation
- Managed by applying and updating the appropriate matrices

Current Transformation Matrix (CTM)

- The CTM is the product of the model-view and the projection matrices
  - produces the required result for subsequent rasterization and fragment processing

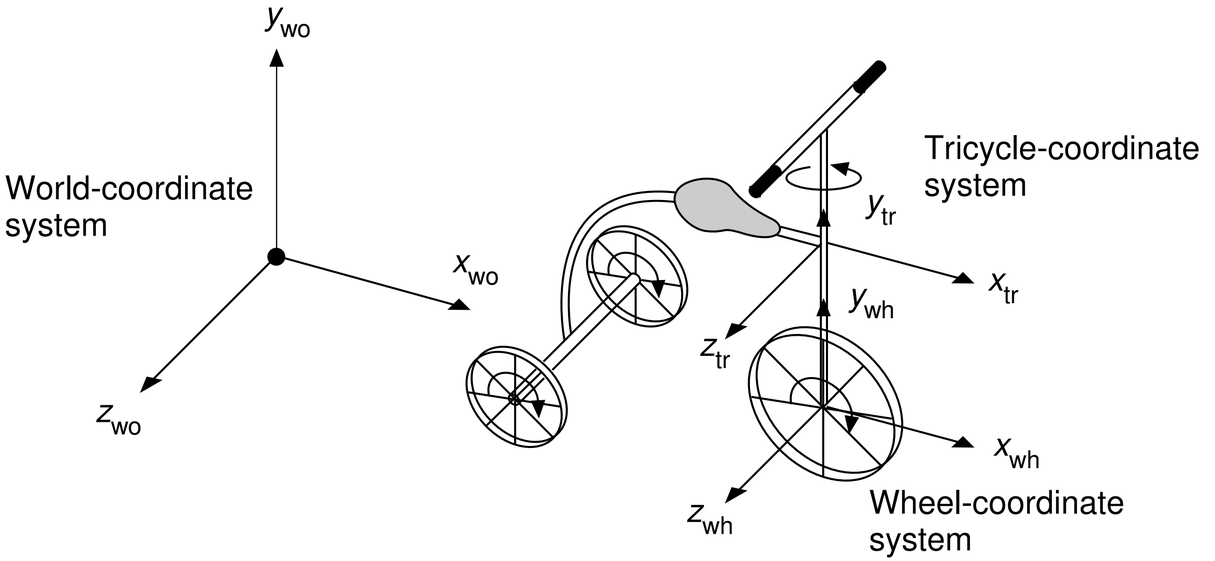


© [AS12] Figure 3.59

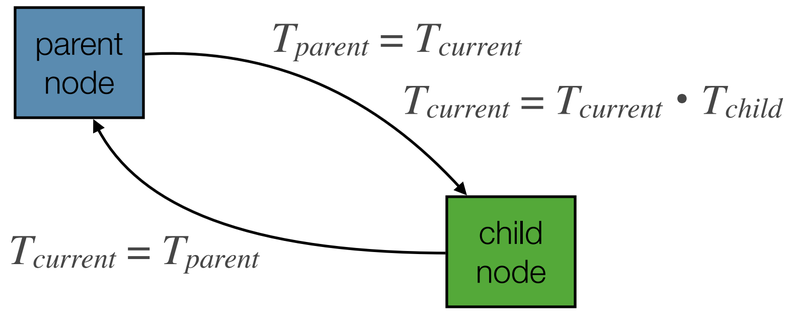
- The model-view transformation may be different for every geometric object or component
  - but the projection from the camera view coordinate system is equal for all geometric elements
  - must maintain the appropriate matrices in the program

CTM and Hierarchical Scene Graphs

- Hierarchical scene graph or articulated models naturally lead to the specification of geometric transformations of subparts relative to their parent components
- Can specify a local transformation in each node relative to its parent node
  - transformation in one node applies to all geometric elements in that subtree
- Maintain the CTM during recursive tree traversal of scene graph
  - save the current transformation in the parent node
  - add child node transformation to the current transformation $T_{current} = T_{current} \cdot T_{child}$
  - restore current transformation again when returning to the parent node



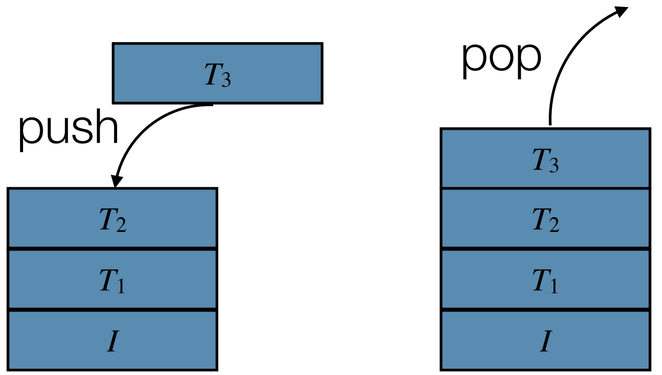
© [AW94] Figure 5.28



Transformation Order and Matrix Stack

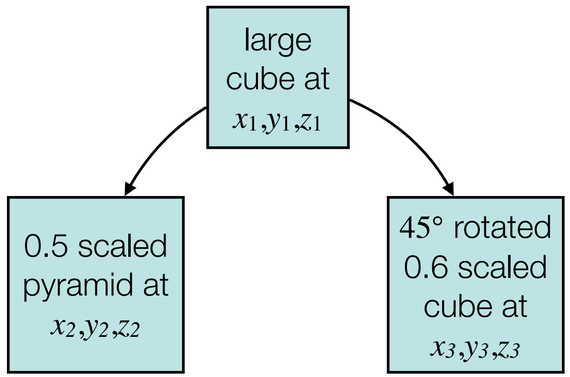
- Postmultiplication of current transformation matrix $T_{current} = T_{current} \bullet T_{new}$ with new transformation T_{new} leads to a classical matrix stack behavior
  - composite transformation $T_{current} = I \bullet T_1 \bullet T_2 \bullet \ldots \bullet T_n$ with components T_k maintained as a stack

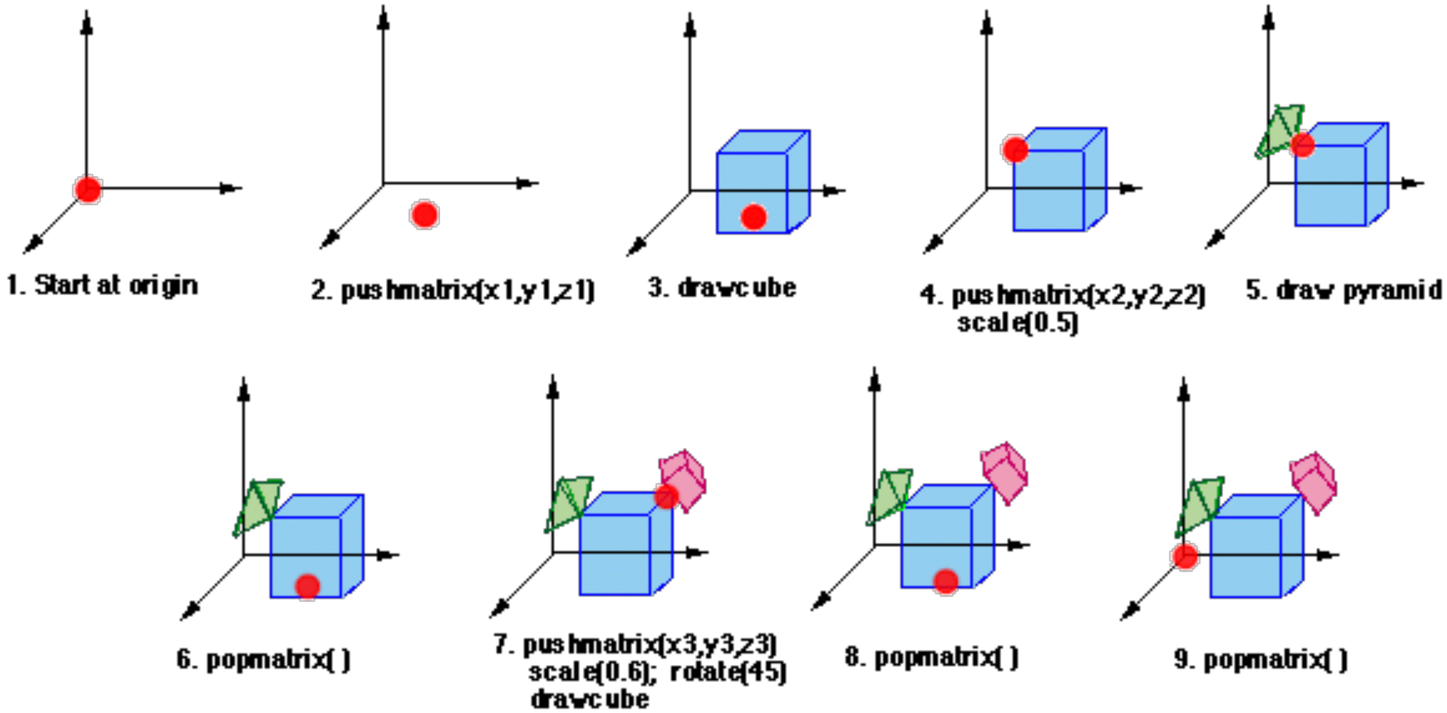
- **Last** specified transformation matrix T_n is actually the **first** being applied to the geometry
  - as if pushed onto a stack for setting up the transformations, and popped from the stack for applying them to the object



Use of Matrix Stack

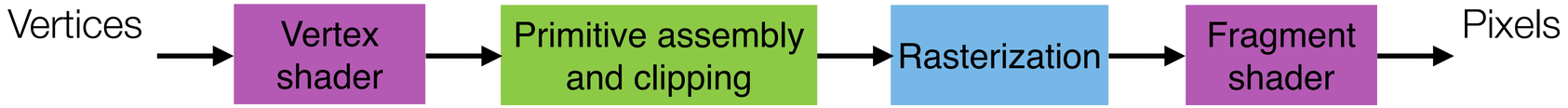
- Managing the current transformation on a stack allows for managing and applying transformations locally to individual objects or parts
  - relative to the parent node in hierarchical models
- And globally to groups of objects





Source https://www.cosc.brocku.ca/Offerings/3P98/course/lectures/2d_3d_xforms/

Review Programmable Shaders



- Initialize and compile a shader from a given source code
  - read shader from file and compile it using `glCreateShader()`, `glShaderSource()` and `glCompileShader()`
- Initialize shader program to be used for rendering
  - create program and attach vertex/fragment shaders using `glCreateProgram()`, `glAttachShader()` and `glLinkProgram()`
- For rendering enable shader to be used
  - activate shader program using `glUseProgram()`

Review Vertex Shader Attribute Locations

- Main program (CPU-side) must make sure the right data is matched to the correct shader input variables (GPU-side)
  - ```
    // per-vertex attribute data in the vertex shader
    in vec3 in_position;
    in vec3 in_color;
    ```

- Establish mapping of the shader input variables to some specific “location” after the initialization of the shader program
  - after the `glAttachShader()` and `glLinkProgram()` calls
  - ```
    // set the locations for the vertex data as used by the VAO
    GLuint vertexLoc, colorLoc, normalLoc;
    vertexLoc = glGetAttribLocation(shaderProgram, “in_position");
    colorLoc = glGetAttribLocation(shaderProgram, “in_color");
    ```

Vertex Transformation Shader

- Simple vertex shader transforming the geometry to the camera, respectively the clip coordinates, and simply forwarding the color information
  - ```
    // minimal input per-vertex attribute data
    in vec3 in_position;
    in vec3 in_color;

    // output color to fragment shader
    out vec4 v_color;

    // transformation matrices
    uniform mat4 view_matrix, proj_matrix;

    void main()
    {
        // apply model-view-projection transform
        gl_Position = proj_matrix * view_matrix * vec4(in_position, 1.0);

        // assign/forward per-vertex color
        v_color = vec4(in_color, 1.0);
    }
    ```
    ```

Review Vertex Shader Attribute Locations

- Map the VBO vertex attribute data to the corresponding shader input variable during initialization of VBO and VAO
  - by use of e.g. the defined `vertexLoc` variable
- The buffers of all vertex attributes must be matched to individual input variables of the vertex shader
  - use a location variable for each attribute like vertex coordinates, colors, normals or texture coordinates

```
// vertex data
float coords[3*numVertices];
void initialize()
{
 // vertex position buffer coordBuffer
  …
  glBufferData(GL_ARRAY_BUFFER, sizeof(coords),
    coords, GL_STATIC_DRAW);
  …
  // vertex array object
  …
  glBindBuffer(GL_ARRAY_BUFFER, coordBuffer);
  glVertexAttribPointer(vertexLoc, 3, GL_FLOAT,
    GL_FALSE, 0,0);
  …
}
```

```
// input variable corresponding to vertexLoc
in vec3 in_position;

void main()
{
  // apply transformation
  gl_Position = matrix * vec4(in_position, 1.0);
  …
}
```

Data Locations for Global Variables

- Other global parameters like rendering parameters or **transformations** remain constant e.g. for one frame and apply to a large group of objects or 3D vertices
  - `uniform` shader variables that are updated just once for a rendered frame
  - or updated for every part of an object or scene graph element

- Also using locations for global variables

```
// get uniform transformation
   matrix variable locations
GLuint viewMatrixLoc =
  glGetUniformLocation(
  shaderProgram, “view_matrix”);
```

```
// input variables
in vec3 in_position;

// uniform variables
uniform mat4 view_matrix;

void main()
{
  // apply transformation
  gl_Position = view_matrix * vec4(in_vertex, 1.0);
  …
}
```

Use of Global Variables

- Update e.g. the transformation matrices from the user input
  - ```
    // update transformation matrix on the CPU
    mat4f viewMatrix = …
    ```
- Activate the applicable shader program
  - ```
    // activate shader program
    glUseProgram(shaderProgram);
    ```
- Map the transformation matrices to the vertex shader
  - ```
    // update the transformation matrix on the GPU
    glUniformMatrix4fv(viewMatrixLoc,  1, false, viewMatrix);

    // update other global uniform variables …
    ```
- Draw the 3D object
    ```

Copyrights

- Many figures in these slides are copyright protected as indicated. Text and all other figures are copyright protected by Renato Pajarola.

- [AS12] Electronic Book Support for Interactive Computer Graphics: A Top-Down Approach with Shader-based OpenGL by Edward Angel and Dave Shreiner. Copyright Addison Wesley 2012.

- [A00] Electronic Book Support for Interactive Computer Graphics: A Top-Down Approach Using OpenGL by Edward Angel. Copyright E. Angel 2000. (http://www.cs.unm.edu/~angel/BOOK/SECOND_EDITION/)

- [AW94] Electronic Instructors Manual (EIM) for Introduction to Computer Graphics by James Foley, Andries van Dam, Steven Feiner, John Hughes, and Richard Phillips. Copyright Addison Wesley 1994.

- [AC97] Learning Rendering CD accompanying 3D Computer Graphics by Alan Watt. Copyright A. Watt and L. Cooper 1997. (Permission for copying and distribution without direct commercial advantage. To copy otherwise, or to republish, requires specific permission.)

- [S02] Electronic Figures for Fundamentals of Computer Graphics by Peter Shirley. Copyright P. Shirley 2002. (http://www.cs.utah.edu/~shirley/fcg/figures/)